

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand

6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I _____M.SHARMILA(192572139)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

```
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Student (
->     student_id INT PRIMARY KEY,
->     name VARCHAR(50) NOT NULL,
->     age INT,
->     email VARCHAR(100) UNIQUE,
->     dept_id INT,
->     FOREIGN KEY (dept_id) REFERENCES Departmen
-> );
ERROR 1046 (3D000): No database selected
mysql>
mysql> SELECT dept_id, COUNT(*)
-> FROM Employee;
ERROR 1046 (3D000): No database selected
mysql> SELECT dept_id, COUNT(*) AS employee_count
-> FROM Employee
-> GROUP BY dept_id;
```

1. CREATE TABLE Employee

Creates a new table named Employee.

2. emp_id INT PRIMARY KEY

Creates the employee ID column. INT stores integer values, and PRIMARY KEY ensures every employee has a unique ID.

3. emp_name VARCHAR(50) NOT NULL

Stores the employee's name up to 50 characters. NOT NULL ensures the name cannot be left empty.

4. salary DECIMAL(10,2)

Stores the employee's salary. It allows up to 10 digits, with 2 digits after the decimal point.

5. email VARCHAR(100) UNIQUE

Stores the employee's email address. UNIQUE prevents duplicate email addresses.

6. dept_id INT

Stores the department ID to which the employee belongs.

7. FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

Creates a relationship between the Employee and Department tables. It ensures that the department ID entered in the Employee table exists in the Department table.

Overall Explanation

This query creates an Employee table with proper data types and constraints. The constraints help maintain data accuracy, uniqueness, and relationships between the Employee and Department tables.

2. Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level

```
ERROR 1046 (3D000): No database selected
mysql> CREATE TABLE Student (
->     student_id INT PRIMARY KEY,
->     name VARCHAR(50) NOT NULL,
->     age INT,
->     email VARCHAR(100) UNIQUE,
->     dept_id INT,
->     FOREIGN KEY (dept_id) REFERENCES Depart
-> );
ERROR 1046 (3D000): No database selected
mysql>
mysql> SELECT dept_id, COUNT(*)
-> FROM Employee;
ERROR 1046 (3D000): No database selected
mysql> SELECT dept_id, COUNT(*) AS employee_count
-> FROM Employee
-> GROUP BY dept_id;
```

QUERIES EXPLANATION:

CREATE DATABASE college_db;

Creates a new database named **college_db**.

USE college_db;

Selects the college_db database so that the tables can be created inside it.

CREATE TABLE Department (...)

Creates the **Department** table.

dept_id is the **Primary Key**, so every department has a unique ID.

dept_name stores the department name and NOT NULL makes it compulsory.

CREATE TABLE Student (...)

Creates the Student table.

student_id is the Primary Key.

name cannot be empty because of NOT NULL.

email must be unique because of UNIQUE.

dept_id stores the department ID.

FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

Creates a relationship between the Student and Department tables. It ensures that a student's department ID exists in the Department table.

INSERT INTO Department ...

Adds sample department records such as Computer Science, Information Technology, and Electronics.

3. Correct a faulty SELECT query where column names are misspelled or improperly referenced.

```
mysql> SELECT student_id, name, age
-> FROM Student
-> WHERE department = 'CSE'
-> AND age > 18;

+-----+-----+-----+
| student_id | name   | age |
+-----+-----+-----+
| 101        | Anish  | 19  |
| 103        | Sharmi | 20  |
| 105        | Ashlin | 21  |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

QUERIES EXPLANATION

1. Incorrect Query

```
SELECT student_id, studnt_name, email
```

```
FROM Student;
```

This query tries to retrieve the student ID, student name, and email from the Student table. However, it contains errors because studnt_name and email are not the correct column names.

2. Corrected Query

```
SELECT student_id, name, email
```

```
FROM Student;
```

This query correctly retrieves the required columns from the Student table.

- SELECT is used to retrieve data from a table.
- student_id retrieves the student's ID.
- name retrieves the student's name.
- email retrieves the student's email address.
- FROM Student specifies that the data should be taken from the Student table.

3. Overall Explanation

The original query gives an error because the column names are misspelled.

After replacing studnt_name with name and email with email, the query executes successfully and displays the student details.

Purpose: To identify and correct incorrectly referenced column names in an SQL SELECT query.

4. Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect

1. SELECT Student.name, Department.dept_name

This selects the student's name from the Student table and the department name from the Department table.

2. FROM Student

This specifies that the main data is taken from the Student table.

3. JOIN Department

This joins the Student table with the Department table.

4. ON Student.dept_id = Department.dept_id

This is the **join condition**. It matches the dept_id in the Student table with the corresponding dept_id in the Department table.

4. Identify and correct a subquery error where multiple values are returned instead of a single value explain the queries

```
mysql> SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date
-> FROM Customer c
-> JOIN Orders o;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near ';' at line 3

-- Corrected Query (JOIN condition added)
mysql> SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date
-> FROM Customer c
-> JOIN Orders o
-> ON c.Customer_ID = o.Customer_ID;
+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Order_ID | Order_Date |
+-----+-----+-----+-----+
| 1           | Arun          | 101      | 2024-05-01 |
| 1           | Arun          | 102      | 2024-05-03 |
| 2           | Ravi          | 103      | 2024-05-02 |
| 3           | Sharmi        | 104      | 2024-05-04 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Incorrect Query

SELECT name

```
FROM Student

WHERE dept_id = (

    SELECT dept_id

    FROM Student

);
```

Error

The subquery:

```
SELECT dept_id

FROM Student;
```

returns multiple department IDs because there are several students.

But the = operator expects the subquery to return only one value.

This can cause an error such as:

ERROR 1242 (21000): Subquery returns more than 1 row

Corrected Query

If we want to find students who belong to any of the departments returned by the subquery, use IN instead of =.

```
SELECT name

FROM Student

WHERE dept_id IN (

    SELECT dept_id

    FROM Student

);
```

Explanation of the Queries

1. SELECT name

Retrieves the names of students.

2. FROM Student

Specifies that the data is taken from the Student table.

3. WHERE dept_id IN (...)

Checks whether the student's dept_id matches **any value** returned by the subquery.

4. Subquery

```
SELECT dept_id
```

```
FROM Student;
```

Retrieves the department IDs from the Student table. Since it can return multiple rows, IN is appropriate.

Why IN is Used

= is used when the subquery returns **one value**.

```
WHERE dept_id = (subquery)
```

IN is used when the subquery can return **multiple values**.

```
WHERE dept_id IN (subquery)
```


5. Identify and correct a subquery error where multiple values are returned instead of a single value explain the queries

```
mysql> SELECT name, department_id
-> FROM Employee
-> WHERE department_id = (SELECT department_id
-> FROM Department);
ERROR 1242 (21000): Subquery returns more than 1 row

-- Corrected Query (use IN instead of =)
mysql> SELECT name, department_id
-> FROM Employee
-> WHERE department_id IN (SELECT department_id
-> FROM Department);
+-----+-----+
| name      | department_id |
+-----+-----+
| Anish     | 10             |
| Sharmi    | 10             |
| Ashlin    | 20             |
| Aravind   | 20             |
+-----+-----+
4 rows in set (0.00 sec)
mysql>
```

1. CREATE TABLE Employee

Creates a new table named **Employee**.

2. emp_id INT PRIMARY KEY

Creates the employee ID column. INT stores integer values, and PRIMARY KEY ensures every employee has a unique ID.

3. emp_name VARCHAR(50) NOT NULL

Stores the employee's name up to 50 characters. NOT NULL ensures the name cannot be left empty.

4. salary DECIMAL(10,2)

Stores the employee's salary. It allows up to 10 digits, with 2 digits after the decimal point.

5. email VARCHAR(100) UNIQUE

Stores the employee's email address. UNIQUE prevents duplicate email addresses.

6. dept_id INT

Stores the department ID to which the employee belongs.

7. FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

Creates a relationship between the Employee and Department tables. It ensures that the department ID entered in the Employee table exists in the Department table.

6. Debug a VIEW creation statement that contains invalid column references or syntax errors.

```
mysql> CREATE VIEW student_info AS
  -> SELECT stud_id, full_name, agee, department
  -> FROM Student;
ERROR 1054 (42S22): Unknown column 'agee' in 'field list'

-- Corrected VIEW creation statement
mysql> CREATE VIEW student_info AS
  -> SELECT stud_id, full_name, age, department
  -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

-- Display data from the VIEW
mysql> SELECT * FROM student_info;
+-----+-----+-----+-----+
| stud_id | full_name | age | department |
+-----+-----+-----+-----+
| 101     | Anish     | 19  | CSE        |
| 102     | Sharmi    | 20  | CSE        |
| 103     | Ashlin    | 21  | ECE        |
| 104     | Aravind   | 22  | EEE        |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> █
```

1. CREATE VIEW StudentDetails AS

Creates a virtual table called **StudentDetails**. A view stores the SQL query rather than storing separate data.

2. SELECT Student.student_id, Student.name, Department.dept_name

Selects the required columns:

- student_id → student's ID.

- name → student's name.
- dept_name → department name.

The table names are written before the column names to clearly identify which table they belong to.

3. FROM Student

Specifies that the main data comes from the Student table.

4. JOIN Department

Combines the Student table with the Department table.

5. ON Student.dept_id = Department.dept_id

Specifies the correct relationship between the two tables. It matches the department ID in Student with the department ID in Department.

7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

```
mysql> CREATE VIEW student_info AS
-> SELECT stud_id, full_name, agee, department
-> FROM Student;
ERROR 1054 (42S22): Unknown column 'agee' in 'field list'

-- Corrected VIEW creation statement
mysql> CREATE VIEW student_info AS
-> SELECT stud_id, full_name, age, department
-> FROM Student;
Query OK, 0 rows affected (0.01 sec)

-- Display data from the VIEW
mysql> SELECT * FROM student_info;
+-----+-----+-----+-----+
| stud_id | full_name | age | department |
+-----+-----+-----+-----+
| 101     | Anish     | 19  | CSE         |
| 102     | Sharmi    | 20  | CSE         |
| 103     | Ashlin    | 21  | ECE         |
| 104     | Aravind   | 22  | EEE         |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

QUERIES EXPLANTION

Unnecessary Query

SELECT *

FROM Student

WHERE dept_id = 10

AND dept_id = 10

AND name IS NOT NULL;

Problem

The condition:

dept_id = 10

is written **twice**, which is unnecessary. Repeating the same condition does not change the result.

Optimized Query

SELECT *

FROM Student

WHERE dept_id = 10

AND name IS NOT NULL;

Explanation of the Queries

1. SELECT *

Retrieves all columns from the Student table.

2. FROM Student

Specifies that the data should be retrieved from the Student table.

3. WHERE dept_id = 10

Selects only students who belong to department **10**.

4. AND name IS NOT NULL

Ensures that only students whose name has a value are selected.

Why the Query is Optimized

The original query checks dept_id = 10 twice:

8. Identify violations of integrity constraints in a given table and correct them. EXPLAIN THE QUERIES

Identifying and Correcting Integrity Constraint Violations

Bloom's Level: BL3 – Apply

Example Table

Suppose we have the following Student table:

```
CREATE TABLE Student (  
  
    student_id INT PRIMARY KEY,  
  
    name VARCHAR(50) NOT NULL,  
  
    email VARCHAR(100) UNIQUE,  
  
    dept_id INT  
  
);
```

1. Primary Key Violation

Incorrect Query:

```
INSERT INTO Student  
  
VALUES (101, 'Anish', 'anish@gmail.com', 10)  
  
INSERT INTO Student  
  
VALUES (101, 'Sharmi', 'sharmi@gmail.com', 20);
```

Here, student_id = 101 is entered twice.

Error:

ERROR: Duplicate entry '101' for key 'PRIMARY'

Correct Query:

```
INSERT INTO Student  
  
VALUES (102, 'Sharmi', 'sharmi@gmail.com', 20);
```

Explanation:

student_id is the **Primary Key**, so every student must have a unique ID. We change the second ID from 101 to 102.

2. NOT NULL Violation

Incorrect Query:

```
INSERT INTO Student
```

```
VALUES (103, NULL, 'ashlin@gmail.com', 10);
```

Error:

The name column cannot contain NULL because it has the NOT NULL constraint.

Correct Query:

```
INSERT INTO Student
```

```
VALUES (103, 'Ashlin', 'ashlin@gmail.com', 10);
```

Explanation:

A valid student name must be provided instead of NULL.

3. UNIQUE Constraint Violation

Incorrect Query:

```
INSERT INTO Student
```

```
VALUES (104, 'Arun', 'anish@gmail.com', 10);
```

If anish@gmail.com already exists, this violates the UNIQUE constraint.

Correct Query:

```
INSERT INTO Student
```

```
VALUES (104, 'Arun', 'arun@gmail.com', 10);
```

Explanation:

The email column must contain a unique value for every student.

9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies

```
mysql> CREATE VIEW student_info AS
  -> SELECT stud_id, full_name, agee, department
  -> FROM Student;
ERROR 1054 (42S22): Unknown column 'agee' in 'field list'

-- Corrected VIEW creation statement
mysql> CREATE VIEW student_info AS
  -> SELECT stud_id, full_name, age, department
  -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

-- Display data from the VIEW
mysql> SELECT * FROM student_info;
+-----+-----+-----+-----+
| stud_id | full_name | age | department |
+-----+-----+-----+-----+
| 101     | Anish     | 19  | CSE        |
| 102     | Sharmi    | 20  | CSE        |
| 103     | Ashlin    | 21  | ECE        |
| 104     | Aravind   | 22  | EEE        |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

1. Relation Violating 1NF

Suppose we have this table: The Phone_Numbers column contains multiple values in one cell. This violates First Normal Form (1NF) because every column should contain only one atomic value.

Corrected Table

CREATE TABLE Student (

student_id INT PRIMARY KEY,

name VARCHAR(50) NOT NULL

);

CREATE TABLE StudentPhone (

student_id INT,

phone_number VARCHAR(15),

PRIMARY KEY (student_id, phone_number),

FOREIGN KEY (student_id) REFERENCES Student(student_id)

);

Explanation

CREATE TABLE Student

Creates the main Student table.

student_id INT PRIMARY KEY

Creates a unique ID for every student.

CREATE TABLE StudentPhone

Creates a separate table for phone numbers.

PRIMARY KEY (student_id, phone_number)

Creates a **composite primary key**, ensuring that the same phone number is not unnecessarily repeated for the same student.

FOREIGN KEY (student_id) REFERENCES Student(student_id)

Connects the StudentPhone table with the Student table.

Now each phone number is stored separately, satisfying **1NF**.

10. Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.


```

mysql> CREATE VIEW student_info AS
  -> SELECT stud_id, full_name, agee, department
  -> FROM Student;
ERROR 1054 (42S22): Unknown column 'agee' in 'field list'

-- Corrected VIEW creation statement
mysql> CREATE VIEW student_info AS
  -> SELECT stud_id, full_name, age, department
  -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

-- Display data from the VIEW
mysql> SELECT * FROM student_info;
+-----+-----+-----+-----+
| stud_id | full_name | age | department |
+-----+-----+-----+-----+
| 101     | Anish     | 19  | CSE         |
| 102     | Sharmi    | 20  | CSE         |
| 103     | Ashlin    | 21  | ECE         |
| 104     | Aravind   | 22  | EEE         |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>

```

Redundant attributes are columns that store the same information repeatedly. This can cause **data duplication, storage wastage, and update inconsistencies**.

For example, in a Student relation, Student_ID, Student_Name, Dept_ID, Dept_Name, and HOD may be stored together. If several students belong to the same department, the Dept_Name and HOD values are repeated for every student.

Identifying Redundancy

Dept_Name and HOD are redundant because they depend on Dept_ID, not directly on Student_ID.

This can lead to problems such as:

- **Update anomaly:** Changing an HOD requires updating multiple records.
- **Insertion anomaly:** A department cannot be added easily if there are no students.
- **Deletion anomaly:** Deleting the last student of a department may also remove the department information.

Applying Normalization

To achieve 3NF, the relation is divided into separate relations:

Student Relation:

- Student_ID
- Student_Name
- Dept_ID

Department Relation:

- Dept_ID
- Dept_Name
- HOD

Dept_ID is used as the link between the two relations.

Why This is 3NF

- **1NF:** Each attribute contains a single, atomic value.
- **2NF:** There are no partial dependencies on a part of a composite key.
- **3NF:** Non-key attributes do not depend on other non-key attributes.